

Executive Metrics

Initial thoughts

Table of Contents

Table of Contents.....	1
Background.....	1
The challenges of 1-1s in today's work setting.....	1
How XXX addresses these challenges.....	2
Executive Metrics.....	2
How would this work?.....	2
Inputs.....	3
Outputs.....	3
Visualized Flow of Goals and Statuses.....	3
Processing.....	4
The "Fixed" version of the problem.....	4
The "Dynamic" version of the problem.....	4
A Proposed Multi Agent Architecture.....	5

Background

XXX is a platform to enable employees and their managers reduce the friction of 1-1s. 1-1s are a vital tool in (a) building rapport between manager and subordinate (b) gaining alignment on team/individual tasks and goals (c) channel for employees to feedback to their manager obstacles or learnings during the course of working on said tasks/goals.

The challenges of 1-1s in today's work setting

1. Almost all teams are distributed with a significant portion of the team being remote
2. Employees generally do not know how to utilize 1-1s to engage their manager
3. Not all employees are self managing hence need some structure to periodically align their efforts to team/organizational goals
4. Managers are usually very busy hence are only able to spare perhaps 30mins a week per report for 1-1. This issue is compounded as reports could be in different timezones and have competing meetings hence reducing the practical window when to have 1-1s.
5. Employees generally are too "sucked in" to the tasks at hand and do not consistently take the time to prep for 1-1s to fully utilize their manager as support system
6. Managers have OKRs or KPIs to meet per given timeframe (month/quarter etc) and ensuring everyone in their team is working "in step" is very challenging.
7. 1-1s and employee performance management are inextricably linked. However, performance reviews are extremely time consuming and are usually not viewed as

time well spent. The reason is that, most of the time, there is little or no trail of thoughts/artifacts/evidence of how employees are working towards their goals and hence it ends up becoming a series of impressions which may bias towards being too positive

How XXX addresses these challenges

1. XXX meets managers and reports in their workflow - Slack, instead of using a separate artifact to lubricate the 1-1 process
2. XXX is “always on” so differences in timezone do not matter
3. XXX will periodically nudge employees to update on goals and flag out obstacles and learnings
4. XXX helps the manager to get a real time assessment of how their team is meeting their objectives (KPIs, OKRs)
5. With the information gathered, XXX is able to create:
 - a. A summary for manager/report for each 1-1 so that it becomes a productive conversation instead of a social event
 - b. A real time report for the manager to assess the state of her team’s progress and what she needs to do to adjust resourcing or step in to assist
 - c. An objective report for regular performance management conversations.

Executive Metrics

The collection of data with all the 1-1 feedback is just the first step in the employee intelligence flywheel. What else can be achieved with existing data collected if augmented with more data + AI?

What if senior executives of a company are able to, on a regular cadence (eg. daily or weekly), view how each team is advancing towards their goals and obstacles are surfaced up early so that they are able to pull in resources to help managers unblock them. This is no different from other operational metrics like revenue, uptime, compute usage costs etc where executives have a means to assess the state of the business. This would absolutely change the game for the need for huge exec level alignment meetings and instead be focused on huddles to address impediments and exploit opportunities.

The analogy is like how the fact that every tank in the US/UK had good radios enabled them to fight better than the Germans in WW2 even though the Germans had superior tank hardware. (see <https://youtu.be/BfTyVnlyzRU>). Because of this, the US/UK armored forces had each tank being a node in an intelligence network. Some units could be reporting back that they were heavily engaged with the enemy (blockers) while a lone tank could possibly observe that there is a gap in the enemy lines that could be exploited (opportunities). This significantly allowed senior leaders to redirect resources as and when needed to support the units on the line.

How would this work?

Note: here we change gears from a high level “where are we going” view to “how can this be built” view

Inputs

1. Cascaded goals - from CEO to L2, L3 ... on to ICs
2. Each individual IC/Manager goals + timestamped status updates
3. Status updates include:
 - a. What was achieved to meet the goal
 - b. What blockers faced but overcome and how
 - c. What blockers faced but need help
 - d. What is interesting that was discovered

Outputs

At each level: L? ... L3, L2, CEO (L1)

1. Team Goal Status
2. Blockers overcome (to be celebrated)
3. Blockers that need escalation
4. What was interesting that was discovered (vetted for the next level eg. CEO doesn't need to know that an engineer found that data structure A did better than data structure B for X task but the CTO/VPE would find it interesting)

Visualized Flow of Goals and Statuses

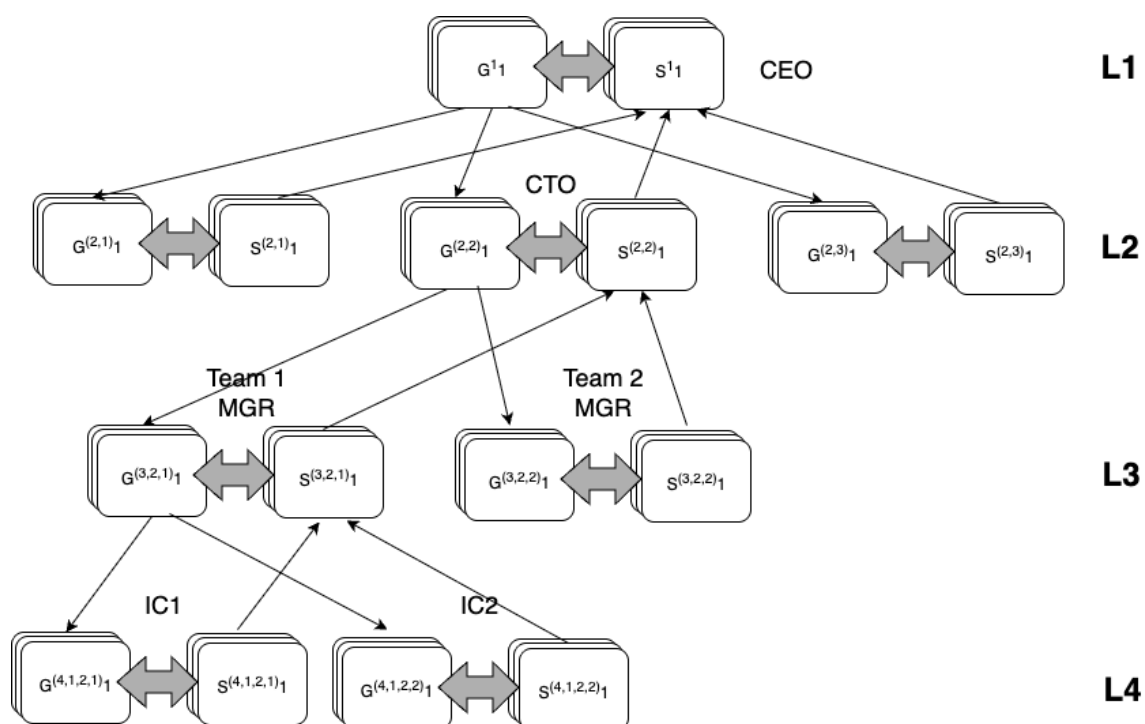


Figure 1: The cascading of goals and statuses

The diagram depicts a 4 level org structure with goals (G) and statuses (S). It also shows how the goals will cascade and the statuses will roll up.

Processing

(this is mostly spitballing)

You would need a mid level model eg. OpenAI 4o to assess at each level:

1. How what was achieved and current unresolved blockers impact status of goal (both quantitatively ie. scoring, as well as qualitatively) and reframe it for consumption at the higher level
2. Scoring the goal (likely an estimate)
3. Reframe for consumption by the next level the blockers that need help to be unblocked. Assessment needs to be made to see if current level manager is able to unblock or it needs to be escalated up

The reason for a mid level model like 4o is that we require some level of nuanced reasoning to perform the processing listed above.

The “interesting” discoveries could be handled by ?? (TBD)

The “Fixed” version of the problem

In this scenario we have all the data as static data. All the statuses would be at the IC level. The task would be to, level by level, perform the assessment of how well the goals at each level are achieved and what the blockers are in a cascade up manner.

The objective of this is to create a framework to be able to:

1. Evaluate the agent architecture on suitability for this task
2. Evaluate the agent architecture in terms of scalability, latency, cost
3. Evaluate how well the agent/prompts/context engineering are performing the task

Key problems so solve:

1. What metrics to use to assess how good the output is (very likely using “LLM as a judge” approach)
2. How to craft the test data that supports various real life status combinations

Once said framework is created, we should be able to iterate and evaluate which agent architecture, prompts, context engineering approach would be best suited for this problem.

The “Dynamic” version of the problem

The dynamic version of the problem is the real problem we are dealing with. Executives all want real time information to make decisions. Business is not static.

The key questions here are:

1. What architecture adjustments would make sense (from the fixed version) to address getting the most up to date information whilst keeping a record of all the updates in status (for traceability)
2. What processing approach would make sense - should we adopt a batch approach ie. process all the cascaded statuses nightly? Or an event based async approach ie. whenever a status gets updated, the relevant agents are activated to cascade up the status?
3. Should goals be modifiable midway thru the assessment cycle?
 - a. If no: what if we work on a goal and realize midway it's not achievable?
 - b. If yes: how does old statuses get dealt with once the goal is modified?
4. (What else?)

A Proposed Multi Agent Architecture

At the very atomic level, we are looking at a multi-agent architecture to perform the following processing:

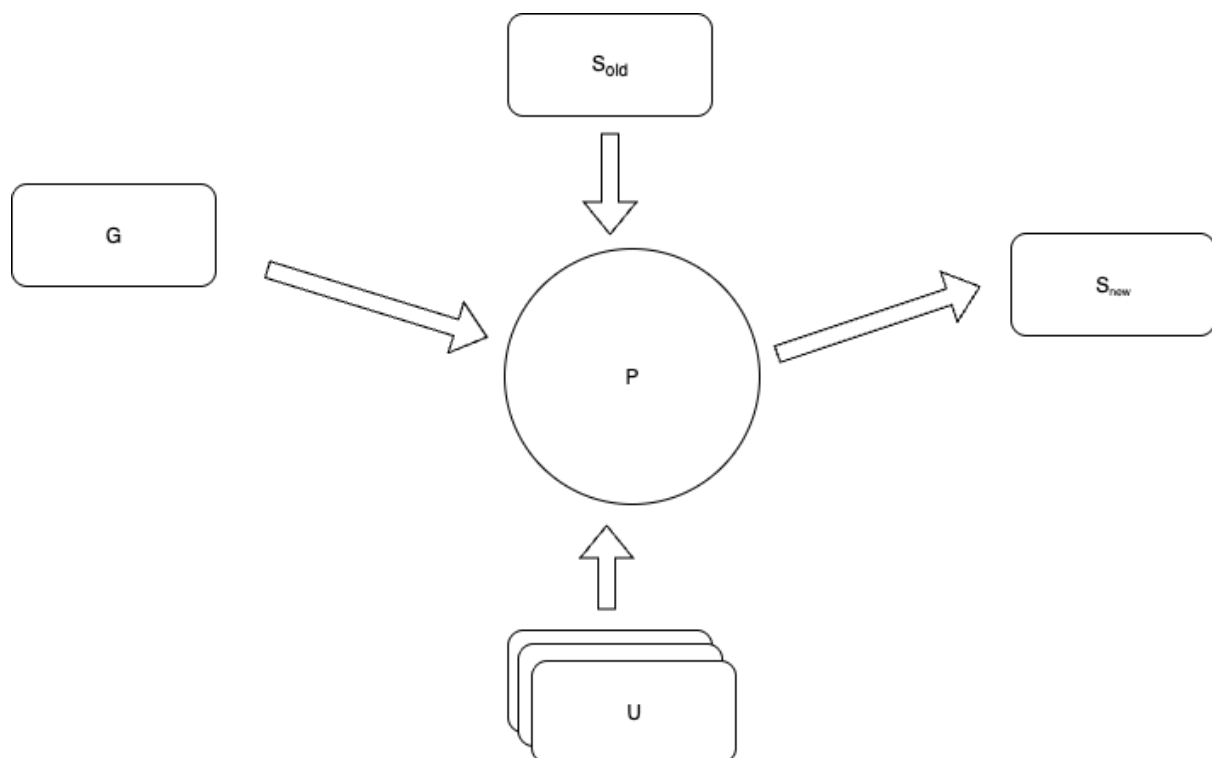


Figure 2: the processing unit at the atomic level

P = the multi agent processor

G = Goal

S_{old} , S_{new} = the status of the Goal before and after the evaluation by P

U = the updates by the reports

For the case of leaf nodes (see Figure 1) ie. at IC level, the updates would be the 1-1 updates that are either (a) what was achieved (b) what blockers need help.

For the case of intermediate nodes ie. manager or jr exec level, the updates would be (a) the Goal+Status of their reports (b) the 1-1 updates with their supervisor

Note: P needs to be able to take on various personalities in order to frame the status and blockers at the right level eg. the CEO is unlikely to find a status like: “The database capacity is currently at 3TB and it will need to be increased by another 2 TB. Also DB latency to be reduced by increasing the connection pool by 5” at a useful level of detail but rather “Some DB tuning is needed. This could require increased CAPEX budget”

The following diagram shows the proposed Multi-Agent Architecture to achieve this

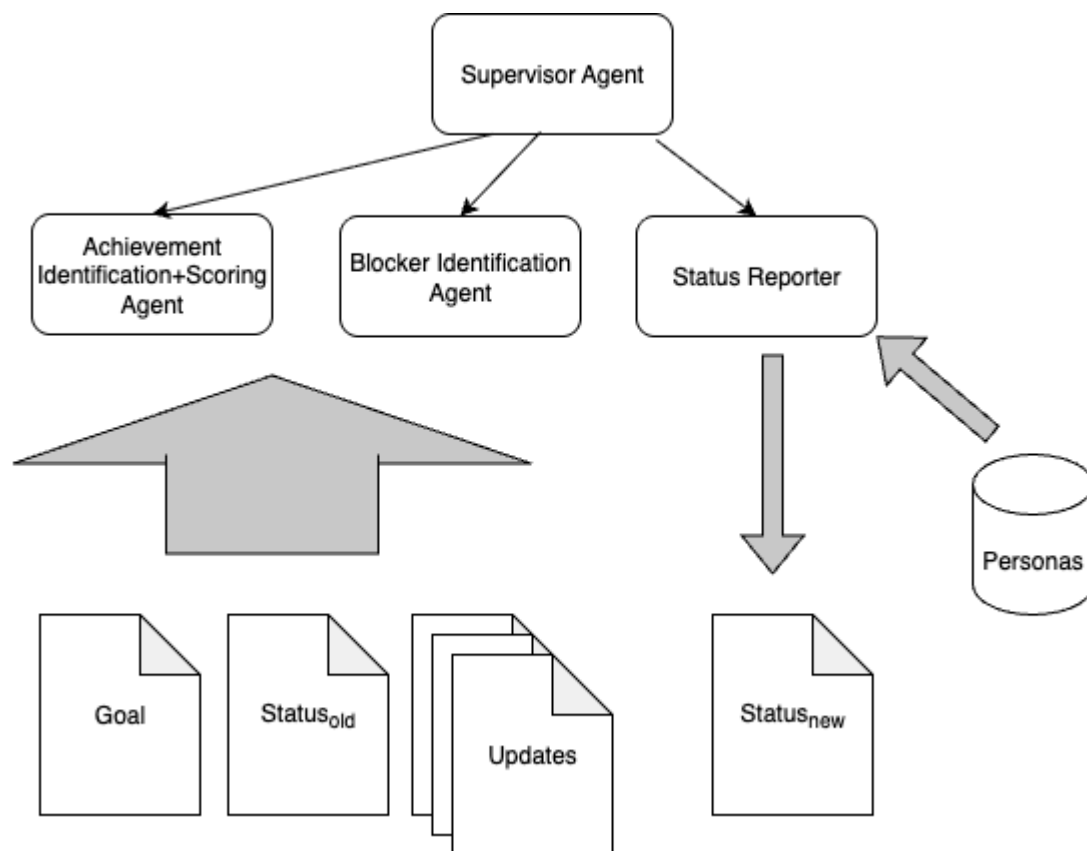


Figure 3: the proposed multi agent architecture

Proposed Toolchain

1. LangGraph (for max control)
2. LangSmith for observability, experimentation and eval
3. Models:
 - a. SaaS (pay by token) - OpenAI 4o (needs eval).
 - b. On prem - ??

Will we need a RAG?

TBD. At this point no but if evals show that we are not getting the right context into the context window for the agents eg. we are seeing moderate to high levels of hallucinations

then it may make sense to and then the issue will be the chunking strategy. If a RAG is needed it'll be for the updates.